

- Update the VFS `super_block` based on these changes  
Accordingly, the error-handling code would need to do the `shut_browsing(info)` and `kfree(info)`. And that would additionally need to go along with the function corresponding to the `kill_sb` function pointer, defined in the previous `real_sfs_bb.c` by `kill_block_super`, called during `umount`. Here are the various code pieces:

```
static int sfs_fill_super(struct super_block *sb, void *data, int
silent) {
    sfs_info_t *info;

    if (!(info = (sfs_info_t *) (kzalloc(sizeof(sfs_info_t), GFP_
KERNEL))))
        return -ENOMEM;
    info->vfs_sb = sb;
    if (init_browsing(info) < 0) {
        kfree(info);
        return -EIO;
    }
    /* Updating the VFS super_block */
    sb->s_magic = info->sb.type;
    sb->s_blocksize = info->sb.block_size;
    sb->s_blocksize_bits = get_bit_pos(info->sb.block_size);

    ~
}

static void sfs_kill_sb(struct super_block *sb) {
    sfs_info_t *info = (sfs_info_t *) (sb->s_fs_info);
    if (info) {
        shut_browsing(info);
        kfree(info);
    }
    kill_block_super(sb);
}
```

Note that `kzalloc()`, in contrast to `kmalloc()`, also zeroes out the allocated location. The `get_bit_pos()` is Pugs' simple way to compute logarithm base 2, as follows:

```
static int get_bit_pos(unsigned int val) {
    int i;

    for (i = 0; val; i++) {
        val >>= 1;
    }
    return (i - 1);
}
```

And `init_browsing()` and `shut_browsing()` are basically the transformations of the earlier user-space functions of `browse_real_sfs.c` into kernel-space code `real_sfs_ops.c`, prototyped in `real_sfs_ops.h`. This basically involves the

following transformations:

- '`int sfs_handle`' into '`sfs_info_t *info`'
- `lseek()` and `read()` into the read from the block device using `sb_bread`
- `calloc()` into `vmalloc()` and then appropriate initialisation by zeros.
- `free()` into `vfree()`

Here's the transformed `init_browsing()` and `shut_browsing()` in `real_sfs_ops.c`:

```
#include <linux/fs.h> /* For struct super_block */
#include <linux/errno.h> /* For error codes */
#include <linux/vmalloc.h> /* For vmalloc, ... */

#include "real_sfs_ds.h"
#include "real_sfs_ops.h"

int init_browsing(sfs_info_t *info) {
    byte_t *used_blocks;
    int i, j;
    sfs_file_entry_t fe;

    if (read_sb_from_real_sfs(info, &info->sb) < 0) {
        return -EIO;
    }
    if (info->sb.type != SIMULA_FS_TYPE) {
        printk(KERN_ERR "Invalid SFS detected. Giving up.\n");
        return -EINVAL;
    }

    /* Mark used blocks */
    used_blocks = (byte_t *) (vmalloc(info->sb.partition_size));
    if (!used_blocks) {
        return -ENOMEM;
    }
    for (i = 0; i < info->sb.data_block_start; i++) {
        used_blocks[i] = 1;
    }
    for (; i < info->sb.partition_size; i++) {
        used_blocks[i] = 0;
    }

    for (i = 0; i < info->sb.entry_count; i++) {
        if (read_from_real_sfs(info, info->sb.entry_table_block_
start, i * sizeof(sfs_file_entry_t), &fe, sizeof(sfs_file_entry_t))
< 0) {
            vfree(used_blocks);
            return -ENOMEM;
        }
        if (!fe.name[0]) continue;
        for (j = 0; j < SIMULA_FS_DATA_BLOCK_CNT; j++) {
            if (fe.blocks[j] == 0) break;
            used_blocks[fe.blocks[j]] = 1;
        }
    }
}
```